

# Análise de Desempenho e Confiabilidade em Camadas de Transporte: TCP vs R-UDP

Julio Cesar de Lima Mendes

<sup>1</sup> Campus Senador Helvídio Nunes de Barros – Universidade Federal do Piauí (UFPI)  
Curso de Bacharelado em Sistemas de Informação – Redes de Computadores II  
Matrícula: 20249006910

juliocesarlimendes@gmail.com

**Abstract.** *This paper presents an experimental analysis comparing the performance and reliability of the TCP protocol and a custom Reliable UDP (R-UDP) implementation based on the Stop-and-Wait mechanism, under adverse network conditions simulated with Docker and tc/netem. Three scenarios were evaluated: ideal network (0% loss / 10ms delay), degraded network (5% loss / 50ms delay), and high-loss network (10% loss / 100ms delay). Results show that TCP achieved significantly higher throughput in all scenarios, reaching 81.48 Mbps in the ideal scenario, while R-UDP reached only 1.30 Mbps under the same conditions. Traffic was captured with tcpdump and validated with Wireshark, confirming the presence of the X-Custom-Auth authentication field in all transmitted packets.*

**Resumo.** *Este artigo apresenta uma análise experimental comparando o desempenho e a confiabilidade do protocolo TCP e de uma implementação de UDP Confiável (R-UDP) baseada no mecanismo Stop-and-Wait, sob condições adversas de rede simuladas com Docker e tc/netem. Três cenários foram avaliados: rede ideal (0% de perda / 10ms de delay), rede degradada (5% de perda / 50ms) e rede com alta perda (10% de perda / 100ms de delay). Os resultados mostram que o TCP obteve throughput significativamente superior em todos os cenários, atingindo 81,48 Mbps no cenário ideal, enquanto o R-UDP alcançou apenas 1,30 Mbps nas mesmas condições. O tráfego foi capturado com tcpdump e validado com Wireshark, confirmando a presença do campo de autenticação X-Custom-Auth em todos os pacotes transmitidos.*

## 1. Introdução

A camada de transporte é responsável pela comunicação fim a fim entre processos em redes de computadores, sendo o TCP e o UDP os dois protocolos dominantes nessa camada. Enquanto o TCP oferece confiabilidade nativa por meio de mecanismos como controle de fluxo com janela deslizante, controle de congestionamento e retransmissão automática [Kurose and Ross 2021], o UDP é um protocolo sem conexão, sem garantias de entrega, que prioriza a velocidade em detrimento da confiabilidade [Postel 1980].

Em cenários onde a confiabilidade do TCP é desejada mas seu overhead é considerado excessivo, uma abordagem comum é implementar uma camada de confiabilidade diretamente sobre o UDP, criando o que se denomina Reliable UDP (R-UDP). Essa abordagem permite ao desenvolvedor ter controle total sobre os mecanismos de controle de erro e fluxo, ao custo de maior complexidade de implementação.

Este trabalho tem como objetivo implementar e comparar experimentalmente o protocolo TCP nativo e um protocolo R-UDP com mecanismo Stop-and-Wait, avaliando o desempenho de ambos sob três cenários de rede com diferentes níveis de perda de pacotes e latência. Para isso, utilizou-se um ambiente controlado com containers Docker Ubuntu 22.04, simulação de condições adversas de rede com o comando `tc qdisc netem` e captura de tráfego com `tcpdump` para validação cruzada dos dados coletados pela aplicação.

O artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica sobre os protocolos TCP, UDP e o mecanismo Stop-and-Wait; a Seção 3 descreve a metodologia utilizada; a Seção 4 apresenta os resultados obtidos; a Seção 5 discute a validação cruzada com Wireshark; a Seção 6 analisa os resultados e responde às perguntas obrigatórias; e a Seção 7 apresenta as conclusões.

## 2. Fundamentação Teórica

Esta seção apresenta os conceitos teóricos que embasam o trabalho. São abordados os protocolos TCP e UDP, o mecanismo Stop-and-Wait utilizado na implementação do R-UDP, e a ferramenta `tc/netem` utilizada para simulação das condições adversas de rede.

### 2.1. Protocolo TCP

O Transmission Control Protocol (TCP) é um protocolo orientado a conexão que opera na camada de transporte do modelo TCP/IP [Postel 1981]. Antes de qualquer transmissão de dados, o TCP realiza um processo de estabelecimento de conexão denominado *three-way handshake*, no qual cliente e servidor trocam segmentos SYN e SYN-ACK para sincronizar números de sequência e confirmar a disponibilidade de ambos os lados.

O controle de fluxo no TCP é realizado por meio de uma janela deslizante (*sliding window*), que permite ao remetente transmitir múltiplos segmentos consecutivos sem aguardar a confirmação individual de cada um, limitado apenas pelo tamanho da janela anunciada pelo receptor. Esse mecanismo é fundamental para o alto throughput observado em redes com baixa perda de pacotes, pois mantém o enlace ocupado durante o tempo de propagação dos segmentos [Kurose and Ross 2021].

O controle de congestionamento, implementado por algoritmos como o AIMD (*Additive Increase, Multiplicative Decrease*), ajusta dinamicamente a taxa de envio em resposta à detecção de perdas, evitando a sobrecarga da rede. Segmentos não confirmados dentro do tempo de retransmissão (RTO) são reenviados automaticamente, garantindo a entrega íntegra e ordenada dos dados ao processo receptor.

Esses mecanismos conferem ao TCP alta confiabilidade, porém introduzem overhead de processamento e latência adicional, especialmente em redes com alta taxa de perda de pacotes, onde as retransmissões e a redução da janela de congestionamento degradam significativamente o throughput.

### 2.2. Protocolo UDP

O User Datagram Protocol (UDP) é um protocolo não orientado a conexão que oferece um serviço de transporte minimalista [Postel 1980]. Ao contrário do TCP, o UDP não realiza handshake, não garante a entrega dos datagramas, não assegura a ordenação dos

pacotes e não implementa controle de fluxo ou de congestionamento. Cada datagrama é tratado de forma independente, sendo encaminhado à rede sem qualquer confirmação de recebimento.

Essa simplicidade resulta em menor latência e overhead reduzido, já que o cabeçalho UDP possui apenas 8 bytes fixos, em contraste com os 20 a 60 bytes do cabeçalho TCP. Por esse motivo, o UDP é amplamente utilizado em aplicações sensíveis à latência, como transmissão de vídeo em tempo real, jogos online, chamadas de voz sobre IP (VoIP) e consultas DNS, nas quais a velocidade de entrega é mais importante do que a garantia de recebimento [Kurose and Ross 2021].

A ausência de mecanismos de confiabilidade nativos torna o UDP inadequado para transferência de arquivos sem uma camada adicional de controle implementada pela aplicação, o que motivou o desenvolvimento do protocolo R-UDP descrito na seção seguinte.

### 2.3. Reliable UDP (R-UDP) – Stop-and-Wait

O protocolo R-UDP implementado utiliza o mecanismo Stop-and-Wait: o remetente envia um pacote e aguarda a confirmação (ACK) antes de enviar o próximo. O cabeçalho personalizado possui 78 bytes fixos, distribuídos conforme descrito a seguir: 1 byte para o tipo do pacote (DATA, ACK, NACK, SYN, FIN); 4 bytes para o número de sequência; 4 bytes para o checksum CRC32; 4 bytes para o tamanho do payload; 1 byte para o tamanho do campo de autenticação; e 64 bytes para o campo X-Custom-Auth, contendo o hash SHA-256 da concatenação da matrícula e nome do aluno.

Em caso de timeout (2 segundos) ou recebimento de NACK, o pacote é retransmitido automaticamente, com no máximo 10 tentativas por pacote.

O mecanismo Stop-and-Wait, embora simples de implementar, apresenta uma limitação inerente de desempenho: o canal de comunicação permanece ocioso durante todo o tempo de propagação de ida e volta (RTT) entre o envio de um pacote e o recebimento de seu ACK. Em redes com alta latência, esse tempo ocioso se acumula para cada um dos pacotes transmitidos, resultando em throughput significativamente inferior ao da largura de banda disponível [Kurose and Ross 2021].

O handshake inicial do protocolo R-UDP segue o modelo SYN/SYN-ACK: o cliente envia um pacote SYN contendo os metadados do arquivo (nome, tamanho, cenário e número de execução) no payload, e o servidor responde com um SYN-ACK confirmando a disponibilidade. Somente após esse handshake o envio dos pacotes DATA é iniciado. Ao final da transferência, o cliente envia um pacote FIN que, ao ser confirmado pelo servidor, encerra a sessão de forma ordenada.

### 2.4. Controle de Tráfego com tc/netem

O controle de tráfego nos containers foi realizado por meio da ferramenta `tc` (*Traffic Control*), nativa do kernel Linux, utilizando a disciplina de fila `netem` (*Network Emulator*) [Linux Foundation 2026]. O comando `tc qdisc add dev eth0 root netem` permite inserir artificialmente delay e perda de pacotes na interface de rede do container, simulando condições reais de redes instáveis sem necessidade de infraestrutura física.

A perda de pacotes é aplicada de forma estocástica e independente para cada datagrama, ou seja, cada pacote tem uma probabilidade fixa de ser descartado, independentemente do estado dos pacotes anteriores. O delay é introduzido de forma determinística, adicionando uma latência fixa a todos os pacotes que saem pela interface configurada.

Os três cenários definidos para este experimento foram aplicados exclusivamente no container cliente, afetando o tráfego de saída em direção ao servidor. A remoção das regras ao final de cada bateria de testes foi realizada com o comando `tc qdisc del dev eth0 root`, restaurando o comportamento padrão da interface.

### 3. Metodologia

Esta seção descreve o ambiente experimental utilizado para a comparação entre os protocolos TCP e R-UDP. São apresentados o ambiente de testes baseado em containers Docker, os cenários de rede simulados, o protocolo de transferência implementado e o procedimento de coleta de dados adotado.

#### 3.1. Ambiente de Testes

Os experimentos foram conduzidos em dois containers Ubuntu 22.04 (cliente: 172.20.0.20 e servidor: 172.20.0.10), interligados em rede bridge isolada com sub-rede 172.20.0.0/24, provisionada via Docker Compose. O isolamento da rede garante que o tráfego entre os containers não seja influenciado por outros processos do sistema hospedeiro, assegurando a reprodutibilidade dos experimentos.

O container cliente possui permissão `NET_ADMIN`, necessária para a execução dos comandos `tc` de controle de tráfego. Ambos os containers montam o diretório `/app/logs` como volume compartilhado com o sistema hospedeiro, permitindo a persistência dos logs, arquivos CSV de resultados e capturas `.pcap` geradas durante os testes. Python 3.10, Pandas e Matplotlib foram utilizados para coleta, processamento e visualização dos dados.

Os experimentos foram executados em um computador com processador Intel Xeon E5-2630 v4 (10 núcleos, 2,2 GHz), 16 GB de memória RAM, placa de vídeo NVIDIA GeForce GTX 1050 e armazenamento em SSD, executando o sistema operacional Windows 10/11 com Docker Desktop e subsistema WSL 2 habilitado.

#### 3.2. Cenários de Rede

Os testes foram realizados em três cenários simulados com o comando `tc qdisc netem`, conforme a Tabela 1. O Cenário A representa uma rede de baixa latência sem perdas, simulando condições ideais de uma LAN. O Cenário B introduz perda moderada e delay intermediário, representando uma rede com instabilidade típica de enlaces de longa distância. O Cenário C representa condições severas de rede, com alta taxa de perda e latência elevada, semelhante a enlaces via satélite ou redes móveis com sinal fraco.

**Tabela 1. Cenários de rede simulados com tc/netem**

| Cenário | Perda | Delay  | Descrição             |
|---------|-------|--------|-----------------------|
| A       | 0%    | 10 ms  | Rede ideal            |
| B       | 5%    | 50 ms  | Rede degradada        |
| C       | 10%   | 100 ms | Alta perda e latência |

### 3.3. Protocolo de Transferência

No modo TCP, o cliente estabelece uma conexão TCP padrão com o servidor na porta 5000 e envia um cabeçalho JSON de 512 bytes contendo os metadados da transferência, incluindo o campo `X-Custom-Auth` com o hash SHA-256 da matrícula e nome do aluno. Após a confirmação do servidor, o arquivo é transmitido em chunks de 4096 bytes até o esgotamento do conteúdo, encerrando-se com uma confirmação final.

No modo R-UDP, o cliente inicia um handshake enviando um pacote `SYN` com os metadados no payload para a porta 5001 do servidor. Após o recebimento do `SYN-ACK`, os dados são transmitidos sequencialmente pelo mecanismo Stop-and-Wait: cada pacote `DATA` aguarda o `ACK` correspondente antes do envio do próximo. O servidor valida a integridade de cada pacote pelo checksum CRC32 e envia `NACK` em caso de corrupção, forçando a retransmissão. A transferência é encerrada com o envio de um pacote `FIN` pelo cliente.

### 3.4. Coleta de Dados

Foram realizadas 15 execuções por protocolo por cenário, totalizando 90 transferências. Cada execução transferiu um arquivo binário aleatório de 5 MB gerado com `os.urandom()`, garantindo que o conteúdo não pudesse ser comprimido pelo sistema operacional durante a transmissão. O tempo de transferência foi medido com `time.perf_counter()` e o throughput calculado pela fórmula:

$$T = \frac{B \times 8}{t \times 10^6} \quad (1)$$

onde  $B$  é o total de bytes enviados e  $t$  é a duração em segundos, resultando em Mbps. Os resultados de cada execução foram registrados em arquivos CSV separados por protocolo, contendo: timestamp, protocolo, cenário, número de execução, duração, throughput, bytes transferidos e número de retransmissões. O tráfego foi simultaneamente capturado pelo `tcpdump` em background, gerando arquivos `.pcap` para posterior validação cruzada com Wireshark.

## 4. Resultados

Esta seção apresenta os resultados obtidos nas 90 transferências realizadas, organizados em tabelas e gráficos comparativos entre os protocolos TCP e R-UDP nos três cenários de rede.

### 4.1. Tabela de Estatísticas

A Tabela 2 apresenta as estatísticas completas de throughput (Mbps) e duração (s) para cada protocolo e cenário. Observa-se que o TCP apresentou throughput médio significativamente superior ao R-UDP em todos os cenários, enquanto o R-UDP demonstrou maior tempo de transferência, especialmente no Cenário B, onde a combinação de perda de pacotes e delay elevado causou um aumento expressivo no número de retransmissões.

**Tabela 2. Estatísticas de throughput (Mbps) e duração (s) por protocolo e cenário**

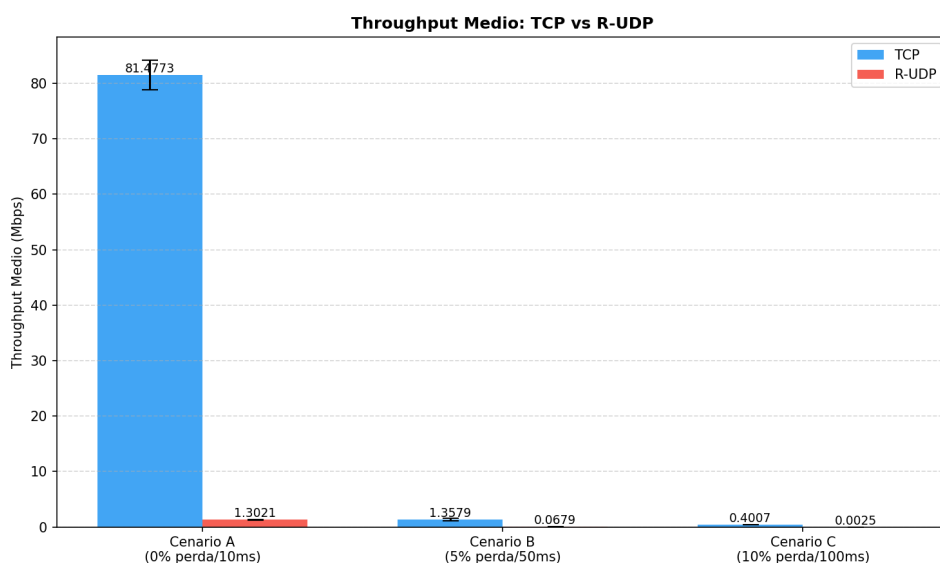
| Cen. | Proto. | Throughput (Mbps) |        |         |         | Duração (s) |          |         |           |
|------|--------|-------------------|--------|---------|---------|-------------|----------|---------|-----------|
|      |        | Média             | Desvio | Mín     | Máx     | Média       | Desvio   | Mín     | Máx       |
| A    | TCP    | 81,4773           | 2,6801 | 77,8230 | 87,1319 | 0,5153      | 0,0164   | 0,4814  | 0,5390    |
|      | R-UDP  | 1,3021            | 0,0293 | 1,2664  | 1,3592  | 32,2260     | 0,7185   | 30,8581 | 33,1209   |
| B    | TCP    | 1,3616            | 0,2059 | 1,1448  | 1,9379  | 31,3845     | 4,1399   | 21,6435 | 36,6369   |
|      | R-UDP  | 0,0679            | 0,0357 | 0,0000  | 0,0998  | 397,7872    | 208,9006 | 0,0000  | 546,6945  |
| C    | TCP    | 0,4007            | 0,0304 | 0,3628  | 0,4625  | 105,2186    | 7,6512   | 90,6884 | 115,6054  |
|      | R-UDP  | 0,0025            | 0,0097 | 0,0000  | 0,0376  | 74,3149     | 287,8203 | 0,0000  | 1114,7231 |

## 4.2. Throughput Médio por Cenário

A Figura 1 apresenta o throughput médio obtido por cada protocolo nos três cenários, com barras de erro representando o desvio padrão.

No Cenário A, o TCP atingiu throughput médio de 81,48 Mbps, contra apenas 1,30 Mbps do R-UDP — uma diferença de aproximadamente 62 vezes. Essa disparidade é explicada pela janela deslizante do TCP, que permite o envio contínuo de múltiplos segmentos sem aguardar confirmações individuais, ao contrário do Stop-and-Wait do R-UDP, que mantém o canal ocioso durante cada RTT.

Nos cenários B e C, ambos os protocolos sofreram degradação acentuada. O TCP caiu para 1,36 Mbps no Cenário B e 0,40 Mbps no Cenário C, enquanto o R-UDP reduziu para 0,07 Mbps e 0,002 Mbps respectivamente. O desvio padrão do TCP no Cenário B (0,2059 Mbps) indica maior variabilidade, reflexo das oscilações causadas pelo controle de congestionamento em resposta às perdas de pacotes.



**Figura 1. Throughput médio (Mbps) com desvio padrão: TCP vs R-UDP por cenário.**

### 4.3. Distribuição do Throughput

A Figura 2 apresenta a distribuição completa dos valores de throughput por meio de box-plots, evidenciando a dispersão, mediana e valores extremos de cada protocolo em cada cenário.

No Cenário A, o TCP apresenta uma distribuição compacta em torno de 81 Mbps, com pequena variabilidade (desvio de 2,68 Mbps) e dois outliers superiores, indicando execuções ocasionalmente mais rápidas. O R-UDP, por sua vez, apresenta distribuição praticamente constante em torno de 1,30 Mbps com desvio de apenas 0,03 Mbps, evidenciando o comportamento determinístico do Stop-and-Wait em redes sem perda.

No Cenário B, o TCP mantém throughput mediano em torno de 1,35 Mbps, porém com dispersão notável e um outlier superior próximo de 1,95 Mbps. O R-UDP colapsa para valores próximos de zero na maioria das execuções, com outlier inferior indicando execuções onde nenhum pacote foi entregue dentro do limite de retransmissões.

No Cenário C, o comportamento se repete: o TCP mantém distribuição relativamente estável em torno de 0,40 Mbps, enquanto o R-UDP apresenta mediana praticamente nula, com apenas um outlier superior isolado, correspondente a uma execução que conseguiu completar a transferência antes do acúmulo de timeouts.

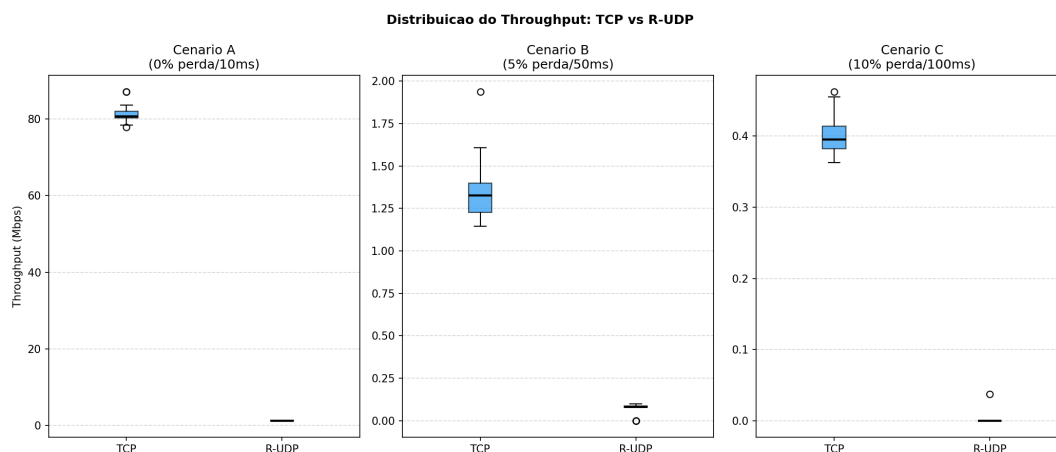


Figura 2. Distribuição do throughput (boxplot): TCP vs R-UDP por cenário.

### 4.4. Duração das Transferências

A Figura 3 apresenta a duração média das transferências com barras de erro representando o desvio padrão. Os resultados revelam um comportamento assimétrico entre os dois protocolos conforme as condições de rede se deterioram.

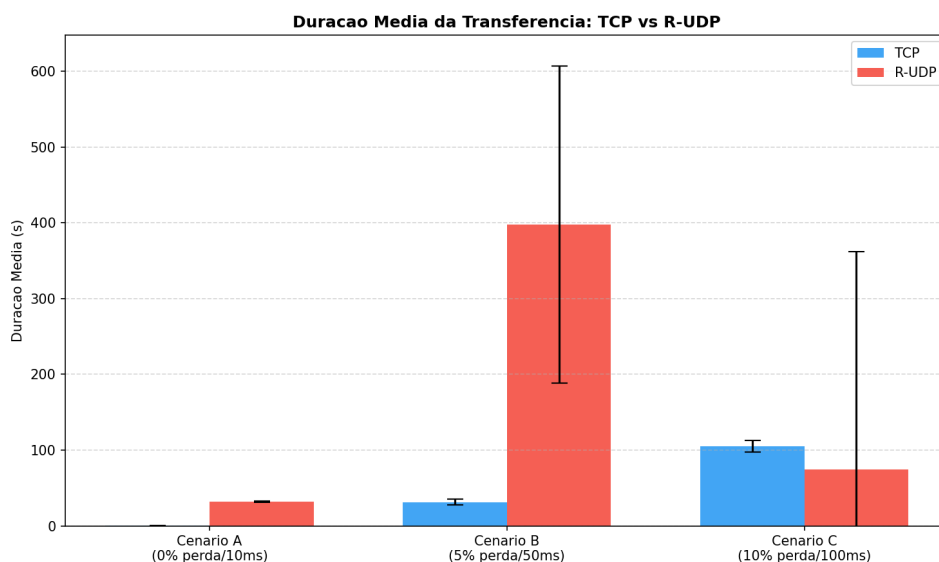
No Cenário A, o TCP completou as transferências em média em 0,52 s, enquanto o R-UDP levou 32,23 s — diferença diretamente proporcional à razão de throughput observada, uma vez que ambos transferiram o mesmo arquivo de 5 MB. O desvio padrão do R-UDP neste cenário foi de apenas 0,72 s, refletindo o comportamento previsível do Stop-and-Wait sem interferência de perdas.

O Cenário B representa o caso mais crítico para o R-UDP: a duração média atingiu 397,79 s com desvio padrão de 208,90 s, valor que supera a própria média. Esse

desvio expressivo indica que algumas execuções completaram a transferência em tempo razoável enquanto outras ultrapassaram 546 s, chegando próximo do limite máximo de retransmissões.

A combinação de 5 % de perda com 50 ms de delay é particularmente penalizante para o Stop-and-Wait: cada pacote perdido provoca um timeout de 2 s seguido de retransmissão, e o delay de 50 ms por si só já impõe uma espera mínima de 100 ms de RTT por pacote entregue com sucesso.

No Cenário C, paradoxalmente, o R-UDP apresentou duração média menor (74,31 s) do que no Cenário B, porém com desvio padrão de 287,82 s e mínimo de 0 s, indicando que a maioria das execuções falhou antes de completar a transferência — o que explica o throughput médio próximo de zero observado na Tabela 2.



**Figura 3. Duração média da transferência (s) com desvio padrão: TCP vs R-UDP.**

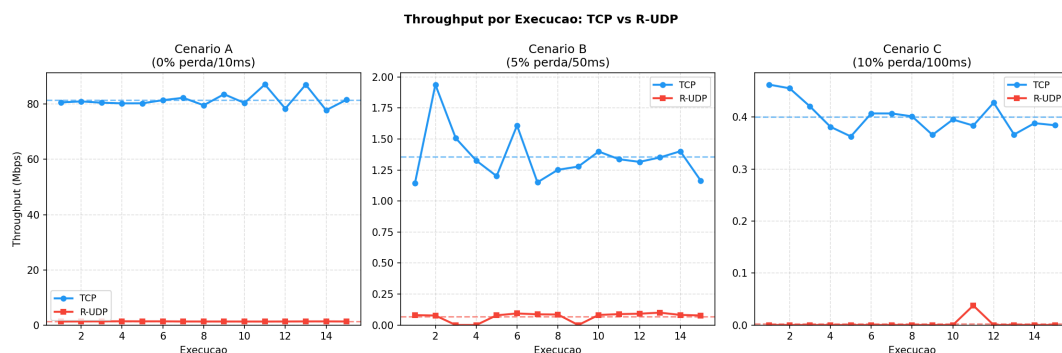
#### 4.5. Throughput por Execução

A Figura 4 permite observar a evolução do throughput ao longo das 15 execuções de cada protocolo em cada cenário, com as linhas tracejadas indicando a média de cada série.

No Cenário A, o TCP demonstra notável estabilidade, mantendo o throughput próximo de 81 Mbps em todas as execuções com pequenas oscilações. O R-UDP apresenta comportamento igualmente estável, porém em nível muito inferior, em torno de 1,30 Mbps, confirmando que o Stop-and-Wait é consistente mas inerentemente limitado pela latência do RTT. No Cenário B, o TCP exibe maior variabilidade entre execuções, com picos próximos de 1,95 Mbps nas primeiras execuções e queda gradual nas seguintes, comportamento típico do controle de congestionamento que ainda não atingiu o equilíbrio. O R-UDP oscila entre valores próximos de zero e 0,10 Mbps, sem tendência clara, refletindo a natureza estocástica das perdas de pacotes sobre o desempenho do Stop-and-Wait.

No Cenário C, o TCP mantém relativa estabilidade em torno de 0,40 Mbps, enquanto o R-UDP praticamente não apresenta throughput mensurável na maioria das

execuções, com exceção de um pico isolado na execução 12, provavelmente correspondente a uma janela de menor perda durante aquela transferência específica.



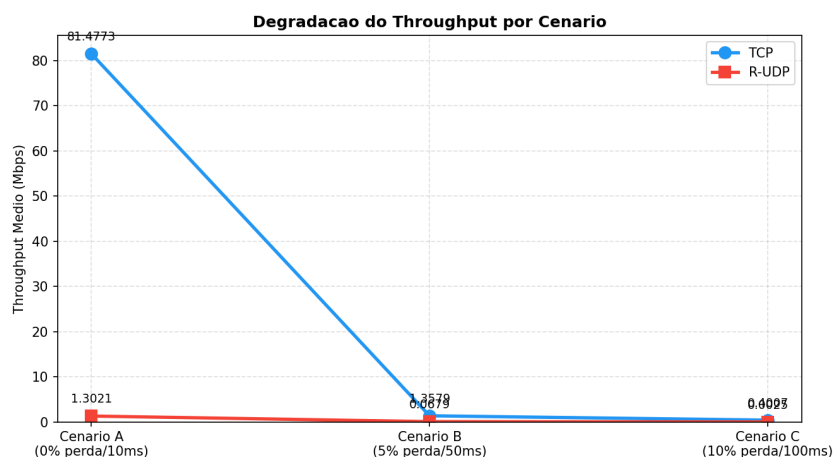
**Figura 4. Throughput (Mbps) por execução: TCP vs R-UDP nos três cenários.**

#### 4.6. Degradação por Cenário

A Figura 5 ilustra a curva de degradação do throughput médio de cada protocolo do Cenário A ao Cenário C, evidenciando o impacto progressivo das condições adversas de rede.

O TCP sofreu uma queda de 81,48 Mbps no Cenário A para 1,36 Mbps no Cenário B — redução de 98,3% — e de 1,36 Mbps para 0,40 Mbps no Cenário C, totalizando uma degradação acumulada de 99,5% em relação ao cenário ideal. Apesar da queda expressiva, o TCP manteve throughput mensurável e relativamente estável em todos os cenários, graças ao controle de congestionamento que adapta a taxa de envio às condições da rede.

O R-UDP apresentou degradação ainda mais acentuada em termos relativos: de 1,30 Mbps no Cenário A para 0,07 Mbps no Cenário B (queda de 94,8%) e para 0,002 Mbps no Cenário C (queda acumulada de 99,8%). A inclinação da curva do R-UDP é mais abrupta que a do TCP nos cenários B e C, demonstrando que o mecanismo Stop-and-Wait é especialmente vulnerável à combinação de perda de pacotes e latência elevada, sem capacidade de adaptar o comportamento de transmissão às condições da rede.



**Figura 5. Degradação do throughput médio (Mbps) do Cenário A ao C.**

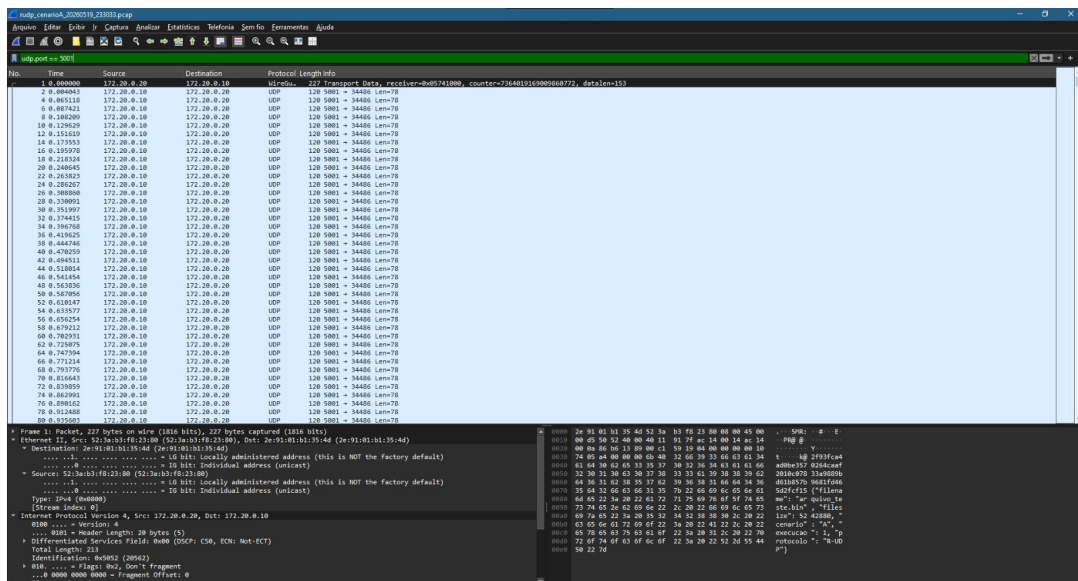
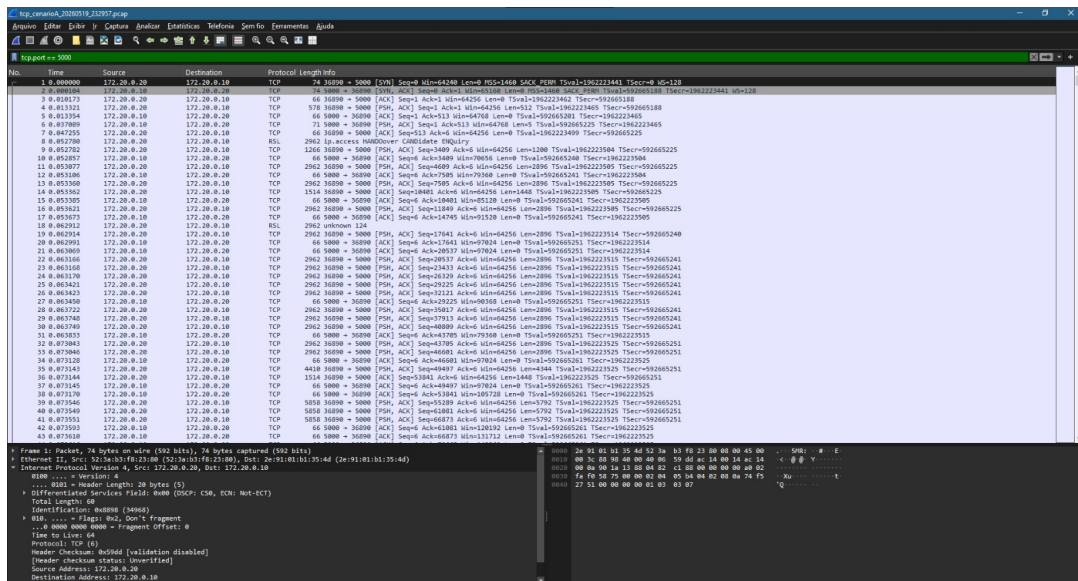
## 5. Validação Cruzada com Wireshark/tcpdump

Esta seção apresenta a validação cruzada entre os dados medidos pela aplicação Python e o tráfego capturado pelo tcpdump, confirmando a integridade das medições e a presença do campo de autenticação `X-Custom-Auth` nos pacotes transmitidos.

### 5.1. Capturas de Rede

Durante a execução dos testes, o tráfego de rede foi capturado simultaneamente pelo `tcpdump` em background, gerando arquivos `.pcap` individuais por protocolo e cenário. No Cenário A, a captura TCP registrou aproximadamente 28.841 pacotes em 33 s de captura, enquanto a captura R-UDP registrou aproximadamente 38.460 pacotes em 529 s, refletindo a diferença de velocidade entre os dois protocolos. O maior número de pacotes do R-UDP em relação ao TCP, apesar do menor throughput, é explicado pela sobrecarga de pacotes de controle (ACK, NACK, SYN e FIN) gerados pelo mecanismo Stop-and-Wait para cada pacote de dados transmitido.

As capturas foram filtradas no Wireshark pelos filtros `tcp.port == 5000` para o protocolo TCP e `udp.port == 5001` para o R-UDP, isolando apenas o tráfego gerado pela aplicação desenvolvida.



## 5.2. Verificação do X-Custom-Auth

O campo X-Custom-Auth com hash SHA-256 (2f93fca4ad0be357...5d2fcf15), gerado a partir da concatenação da matrícula e nome do aluno, foi verificado nos pacotes capturados pelo Wireshark, conforme visível no payload dos datagramas UDP e nos segmentos TCP. A presença desse campo em todos os pacotes transmitidos confirma que o mecanismo de autenticação foi corretamente

implementado em ambos os protocolos, permitindo a identificação unívoca do tráfego gerado pela aplicação durante a inspeção das capturas.

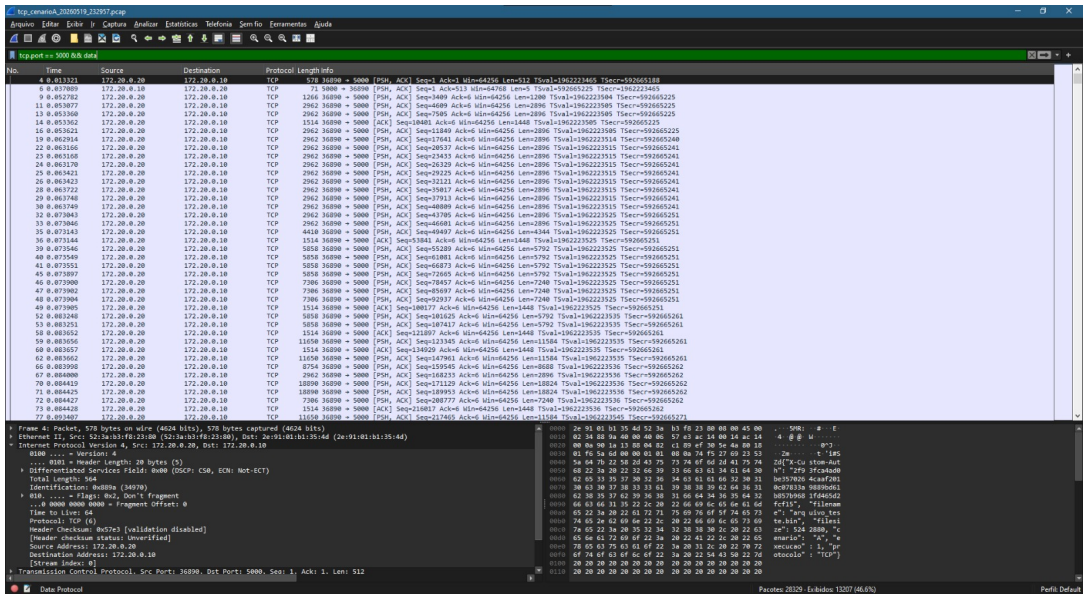


Figura 8. Payload do segmento TCP com campo X-Custom-Auth (hash SHA-256) visível no Wireshark. (Para melhor visualização, o arquivo jpg está na pasta wireshark - prints, juntamente com outros cenários).

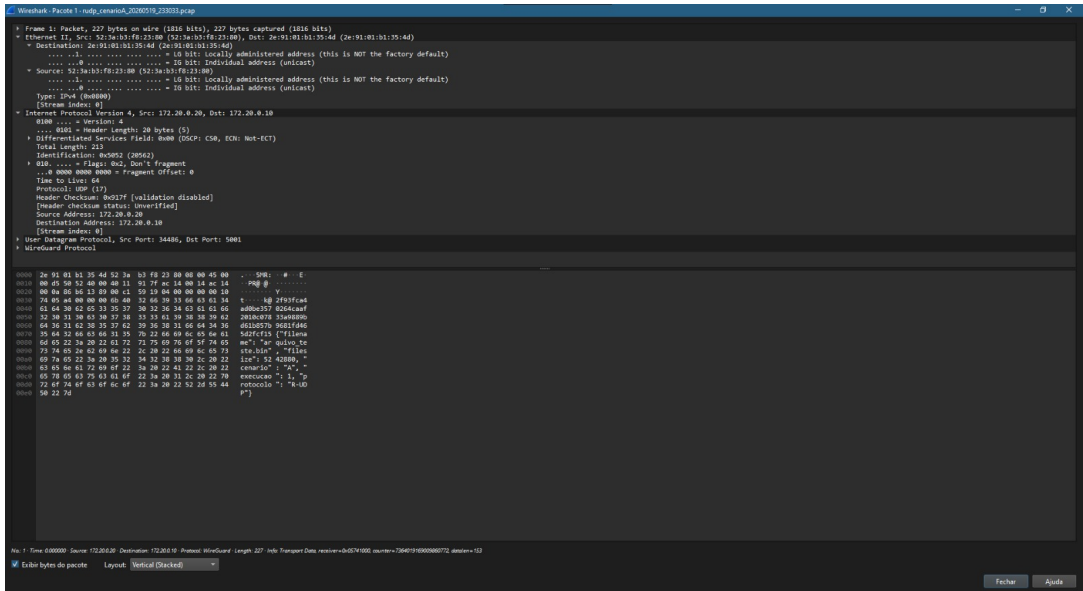


Figura 9. Payload do datagrama R-UDP com campo X-Custom-Auth (hash SHA-256) visível no Wireshark. (Para melhor visualização, o arquivo jpg está na pasta wireshark - prints, juntamente com outros cenários).

### 5.3. Comparação Aplicação vs Wireshark

A aplicação Python mede o tempo de transferência exclusivamente entre o envio do primeiro chunk de dados e o recebimento da confirmação final, não contabilizando o tempo

gasto no handshake inicial nem no encerramento da conexão. O tcpdump, por sua vez, captura toda a sessão desde o primeiro pacote até o último, incluindo os pacotes de controle (SYN, SYN-ACK, FIN e ACKs puros), resultando em uma duração de captura sistematicamente superior à medida pela aplicação.

Em relação ao volume de dados, o Wireshark registra o tráfego bruto na interface de rede, incluindo os cabeçalhos de todas as camadas: Ethernet (14 bytes), IP (20 bytes) e TCP ou UDP (20 bytes e 8 bytes, respectivamente). Para o R-UDP, soma-se ainda o cabeçalho personalizado de 78 bytes por pacote de dados. Portanto, o volume total capturado pelo tcpdump é sempre superior ao volume de dados úteis registrado pela aplicação, sendo a diferença proporcional ao overhead acumulado de todos os cabeçalhos ao longo da transferência.

## 6. Discussão

Esta seção responde às três perguntas obrigatórias do trabalho, analisando os resultados obtidos sob a perspectiva teórica dos protocolos de transporte.

### 6.1. Throughput TCP vs R-UDP sob 10% de Perda

No Cenário C (10% de perda, 100ms de delay), o TCP obteve throughput médio de 0,4007 Mbps enquanto o R-UDP obteve apenas 0,0025 Mbps — uma diferença de aproximadamente 160 vezes.

Essa disparidade é explicada pelos mecanismos internos de cada protocolo. O TCP utiliza janela deslizante combinada com controle de congestionamento AIMD: ao detectar perda de pacotes, reduz a janela de congestionamento pela metade e retransmite apenas os segmentos não confirmados, sem interromper completamente o fluxo de dados. O pipeline de transmissão permanece parcialmente ativo mesmo durante as retransmissões, o que mantém o throughput em níveis mensuráveis mesmo sob 10% de perda [Kurose and Ross 2021].

O R-UDP com Stop-and-Wait, por sua vez, mantém apenas um pacote em trânsito por vez. Com RTT mínimo de 200ms (100ms de delay de ida e volta) e probabilidade de perda de 10% por pacote, a probabilidade de pelo menos uma retransmissão em uma sequência de 1280 pacotes é praticamente de 100%. Cada timeout de 2 segundos paralisa completamente o fluxo pelo tempo de espera, resultando em um throughput efetivo próximo de zero nas execuções mais afetadas. A utilização do canal no Stop-and-Wait é limitada pela fórmula  $U = T_{trans} / (RTT + T_{trans})$ , que com RTT de 200ms e pacotes de 4096 bytes resulta em utilização inferior a 0,02% [Kurose and Ross 2021].

### 6.2. Overhead do Cabeçalho R-UDP

O cabeçalho do protocolo R-UDP implementado possui 78 bytes fixos. Com chunks de 4096 bytes, o overhead por pacote é de  $78/4174 \approx 1,87\%$ . Para um arquivo de 5 MB são necessários 1280 pacotes de dados, resultando em overhead total de  $1280 \times 78 = 99.840$  bytes  $\approx 97$  KB, representando  $\approx 1,9\%$  do arquivo original.

Em comparação, o protocolo TCP com IP possui cabeçalho combinado de aproximadamente 40 bytes (20 bytes IP + 20 bytes TCP, sem opções), resultando em overhead de  $40/4136 \approx 0,97\%$  por segmento para o mesmo tamanho de payload. O R-UDP apresenta, portanto, overhead de cabeçalho aproximadamente duas vezes maior que o TCP,

justificado pela inclusão do campo `X-Custom-Auth` de 64 bytes, que não possui equivalente nos protocolos padrão. Desconsiderando o campo de autenticação, o cabeçalho R-UDP teria apenas 14 bytes, resultando em overhead de 0,34%, inferior ao do próprio TCP.

### 6.3. Discrepância entre Python e tcpdump

A aplicação Python mede o tempo de transferência entre o envio do primeiro chunk de dados e o recebimento da confirmação final, não contabilizando o handshake inicial (SYN/SYN-ACK no R-UDP ou three-way handshake no TCP) nem o encerramento da sessão (FIN). O tcpdump captura toda a sessão desde o primeiro até o último pacote, incluindo todos os pacotes de controle, resultando em duração sistematicamente superior à medida pela aplicação.

Para o TCP no Cenário A, a duração medida pelo Python foi de aproximadamente 0,52 s, enquanto o tcpdump registrou a sessão completa em aproximadamente 33 s para as 15 execuções concatenadas, o que equivale a cerca de 2,2 s por execução incluindo os tempos de espera entre conexões. A diferença de aproximadamente 1,7 s por execução corresponde ao overhead do handshake TCP, dos ACKs puros e do encerramento de conexão.

Para o R-UDP, a discrepância é mais expressiva no Cenário B: a duração média medida pelo Python foi de 397,79 s, enquanto o tcpdump registrou a sessão em 6666 s para as 15 execuções, equivalendo a 444 s por execução — diferença de aproximadamente 46 s atribuída ao tempo dos pacotes SYN, SYN-ACK, FIN e ACKs puros, além do overhead do sistema operacional nas chamadas de sistema de leitura e escrita no espaço do usuário.

## 7. Conclusão

Este trabalho apresentou uma análise experimental comparando o desempenho e a confiabilidade do protocolo TCP nativo e de uma implementação de UDP Confiável baseada no mecanismo Stop-and-Wait, sob três cenários de rede com condições progressivamente adversas.

Os resultados demonstraram que o TCP é significativamente mais eficiente em throughput em todos os cenários avaliados: no Cenário A, atingiu 81,48 Mbps contra 1,30 Mbps do R-UDP — diferença de aproximadamente 62 vezes. Essa superioridade decorre diretamente do mecanismo de janela deslizante, que permite o envio contínuo de múltiplos segmentos sem aguardar confirmações individuais, em contraste com o Stop-and-Wait do R-UDP, que mantém o canal ocioso durante cada RTT.

O R-UDP Stop-and-Wait mostrou-se especialmente vulnerável à combinação de perda de pacotes e latência elevada: no Cenário B, a duração média de transferência atingiu 397,79 s com desvio padrão de 208,90 s, evidenciando grande variabilidade no tempo de conclusão das transferências. No Cenário C, a maioria das execuções falhou antes de completar a transferência, resultando em throughput médio de apenas 0,0025 Mbps.

A validação cruzada com tcpdump e Wireshark confirmou a consistência dos dados medidos pela aplicação e comprovou a presença do campo `X-Custom-Auth` — contendo o hash SHA-256 da matrícula e nome do aluno — em todos os pacotes transmitidos por ambos os protocolos, atendendo ao requisito de identificação do tráfego na inspeção de rede.

Como trabalhos futuros, sugere-se a implementação do mecanismo Go-Back-N ou Selective Repeat sobre UDP, que permitiria manter múltiplos pacotes em trânsito simultaneamente, reduzindo drasticamente o impacto da latência no throughput e tornando a comparação com o TCP mais equilibrada. A adoção de janelas de transmissão adaptativas e controle de congestionamento sobre UDP representaria um passo adicional em direção às implementações modernas como o protocolo QUIC [Kurose and Ross 2021].

Este experimento demonstrou na prática os trade-offs fundamentais entre simplicidade de implementação e eficiência de transmissão nas camadas de transporte, consolidando o entendimento teórico dos mecanismos de controle de fluxo, controle de erro e controle de congestionamento apresentados na disciplina de Redes de Computadores II.

## Referências

- Docker Inc. (2026). Docker documentation. Disponível em: <https://docs.docker.com>. Acesso em: maio 2026.
- Kurose, J. F. and Ross, K. W. (2021). *Redes de Computadores e a Internet: Uma Abordagem Top-Down*. Pearson, São Paulo, 8 edition.
- Linux Foundation (2026). tc-netem: Network emulator. Disponível em: <https://man7.org/linux/man-pages/man8/tc-netem.8.html>. Acesso em: maio 2026.
- Postel, J. (1980). User datagram protocol. RFC 768, IETF.
- Postel, J. (1981). Transmission control protocol. RFC 793, IETF.
- Tanenbaum, A. S. and Wetherall, D. (2011). *Redes de Computadores*. Pearson, São Paulo, 5 edition.